

SOFTWARE DELIVERY MODERNIZATION PROJECT

1. SCOPE DESCRIPTION

The objective of this project is to architect and formalize the modernization of the software delivery lifecycle by transitioning to an automated Continuous Integration and Continuous Deployment (CI/CD) ecosystem. The project focuses on driving development velocity, enforcing continuous software quality gates, establishing deployment predictability, and strengthening cross-team collaboration across Engineering, QA, and IT Operations.

2. KEY PROJECT DELIVERABLES

Architecture & Design

- CI/CD Delivery Workflow Design
- GitHub Integration & Hook Strategy
- Jenkins Pipeline Architecture
- Docker Containerization Strategy

Build & Quality Management

- Automated Maven Build Configuration
- SonarQube Quality Gate Strategy
- Deployment Workflow Framework

Governance & Documentation

- Project Management Plan Baseline
- Comprehensive Implementation Docs
- Post-Project Lessons Learned Report

3. SCOPE BOUNDARIES

✓ In Scope

- Project Governance:** Requirement gathering, stakeholder management, schedule tracking, and proactive risk planning.
- CI/CD Architecture:** Source code branching model, automated build triggers, CI orchestration, and deployment pipeline design.
- Quality Engineering:** Static code inspection, build verification rules, and quality gate enforcement.
- Release Management:** Deployment procedures, automated rollback strategy, and release documentation.
- Project Closure:** Final stakeholder evaluation, documentation handoff, and lessons learned synthesis.

✗ Out of Scope

- Infrastructure server provisioning or hardware setup
- Kubernetes cluster setup and production orchestration
- Cloud architecture migration or legacy refactoring
- Feature application code development and business logic
- Performance, load, or stress testing implementation
- Security penetration testing and vulnerability auditing
- Production real-time infrastructure monitoring tools

4. ACCEPTANCE CRITERIA & SUCCESS METRICS

✓ Acceptance Criteria

- Fully documented, end-to-end CI/CD workflow architecture.

🎯 Success Measures

- Measurable reduction in manual deployment activities.

- Standardized, repeatable build and deployment procedures.
- Complete, peer-reviewed project management documentation.
- Identified risk matrix with active mitigation plans.
- Approved implementation roadmap by key stakeholders.

- Enhanced stability and repeatability across deployments.
- Uniform, company-wide release workflow standardization.
- Strengthened cross-functional Dev, QA, and Ops alignment.
- On-time delivery of all planned scope items within schedule.

5. PROJECT CONSTRAINTS & ASSUMPTIONS

Project Constraints	Key Project Assumptions
• Strict, non-negotiable implementation timeframes. • Fixed project budget for tooling and resources. • Resource availability and bandwidth of assigned personnel. • Dependency on third-party integrations and SaaS tooling.	• Team members possess foundational proficiency in Git and software engineering practices. • Necessary CI/CD tools and test environments are accessible. • Stakeholders provide timely feedback during review cycles. • Project scope remains stable post baseline approval.

6. SCOPE GOVERNANCE & CHANGE CONTROL

Scope Change Control Process

To maintain schedule and quality standards, any proposed modification to the approved scope baseline must follow formal change control procedures: 1) Formal impact analysis on project timeline, budget, and risk profile. 2) Review and evaluation with primary project stakeholders. 3) Explicit written approval prior to implementation. 4) Registration in official project records and updated baseline documentation.